

Lecture 11

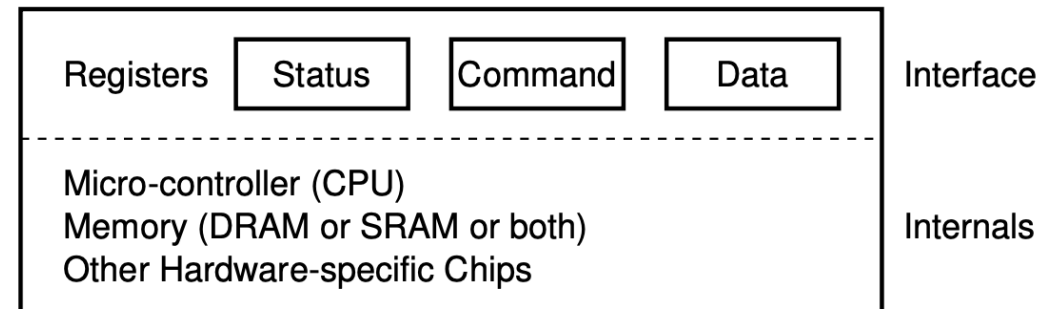
I/O and Storage

Prof. Yinqian Zhang

Spring 2026

A Canonical View of Devices

- Interface
 - The hardware interface a device present to the rest of the system
 - **Status** register: check the current status of the device
 - **Command** register: tell the device to perform a certain task
 - **Data** register: pass data to the device or get data from the device.
- Modern devices (e.g., NVMe SSDs) use multiple command queues (up to 64K deep) instead of a single command register, greatly improving parallelism.



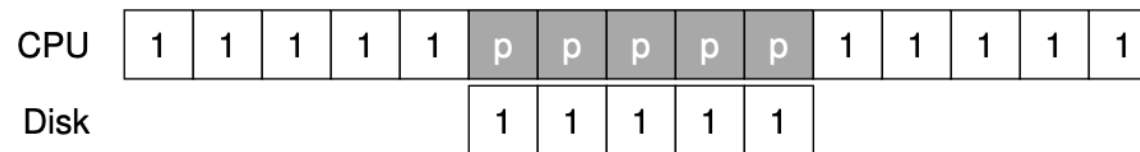
Basic I/O: Polling

- To write a byte of data to device
 - Step 1: OS waits until the device is ready to receive a command by repeatedly reading the status register;
 - Step 2: OS sends some data down to the data register;
 - Step 3: OS writes a command to the command register
 - Step 4: OS waits for the device to finish by again polling it in a loop, waiting to see if it is finished

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

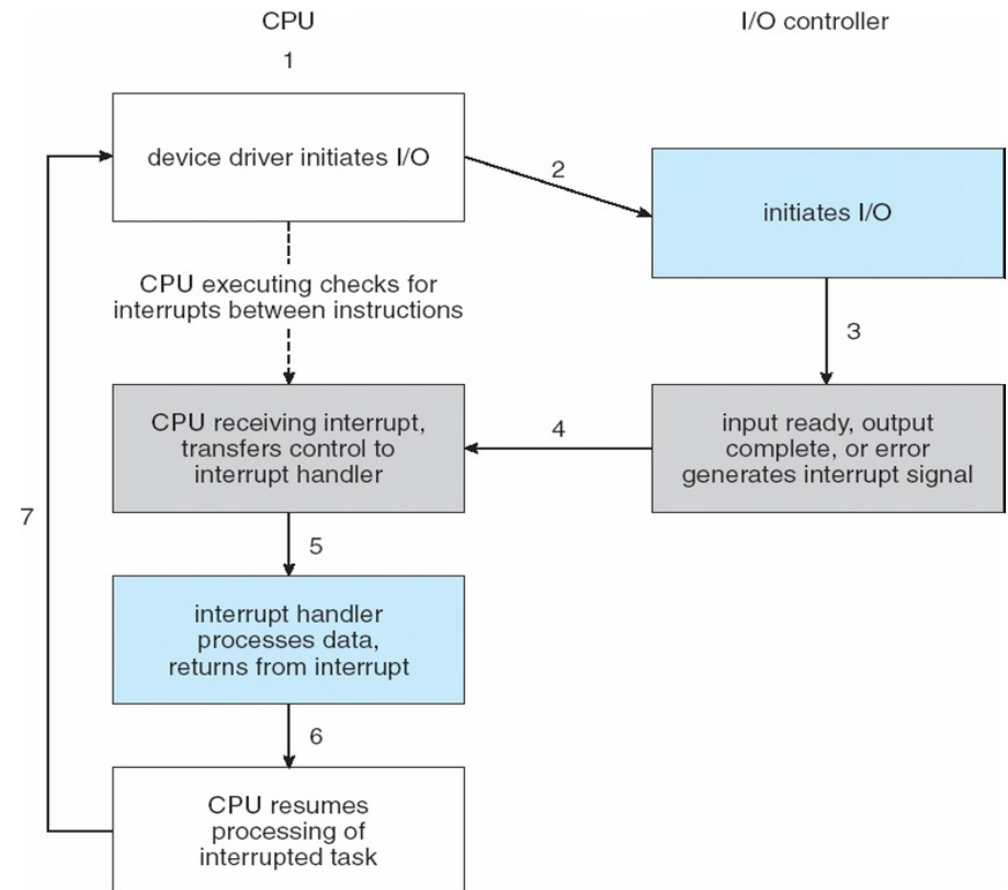
Issues of Polling

- Polling: frequent checking the status of I/O devices
- Polling is inefficient and inconvenient
 - Polling wastes CPU time waiting for slow devices to complete its activity
 - If CPU switches to other tasks, data may be overwritten
 - e.g., keyboard data overflow the buffer



Efficient I/O: Interrupts

- Instead of polling the device repeatedly, the OS can issue a request, put the calling process to sleep, and context switch to another task.
- When the device is finally finished with the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a predetermined **interrupt handler**



Hardware Support for Interrupts

- **Interrupt-request line**, a CPU wire, triggered by I/O device
 - Checked by processor after each instruction
 - Save CPU state and jumps to the interrupt handler
- **Interrupt-controller hardware**
 - Defer interrupt handling during critical processing
 - Dispatch to proper interrupt handler
 - Support multi-level interrupts, high- and low-priority interrupts
- **Modern interrupt controllers:**
 - **APIC** (Advanced Programmable Interrupt Controller) in x86
 - **GIC** (Generic Interrupt Controller) in ARM
 - **PLIC** (Platform-Level Interrupt Controller) or **CLINT** (Core Local Interruptor) in RISC-V systems.

Software Support for Interrupts

- **Interrupt handler** receives interrupts
 - **Maskable** to ignore or delay some interrupts
 - Some are **nonmaskable**, e.g., unrecoverable memory errors.
- A table of **interrupt vectors** to specify interrupt-handling routine
 - Dispatch interrupt to correct handler
 - **Interrupt chaining** if more than one device at the same interrupt number
 - Interrupt handlers on the corresponding chain are called one by one
 - The size of the interrupt table (i.e., number of interrupt vectors) and length of interrupt chains are results of system design trade-off.
 - Priority of interrupts: high-priority interrupts can preempt low-priority interrupts

Programmed I/O

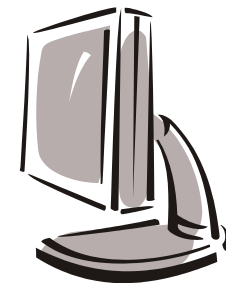
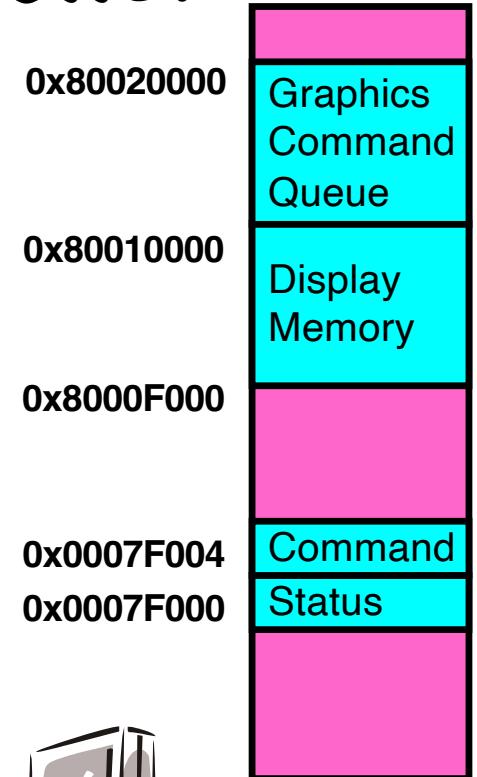
- Explicit I/O instructions
 - in/out instructions on x86:
out 0x21,AL
 - I/O instructions are
privileged instructions
- Memory-mapped I/O
 - Registers/memory appear in
physical address space
 - I/O accomplished with load
and store instructions
 - I/O protection with address
translation

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

Memory-Mapped Display Controller

- Memory-Mapped I/O

- Hardware maps control registers and display memory into physical address space
 - Addresses set by HW jumpers or at boot time
- Simply writing to display memory (also called the “frame buffer”) changes image on screen
 - Addr: 0x8000F000 — 0x8000FFFF
- Writing graphics description to cmd queue
 - Say enter a set of triangles describing some scene
 - Addr: 0x80010000 — 0x8001FFFF
- Writing to the command register may cause on-board graphics hardware to do something
 - Say render the above scene
 - Addr: 0x0007F004



**Physical
Address
Space**

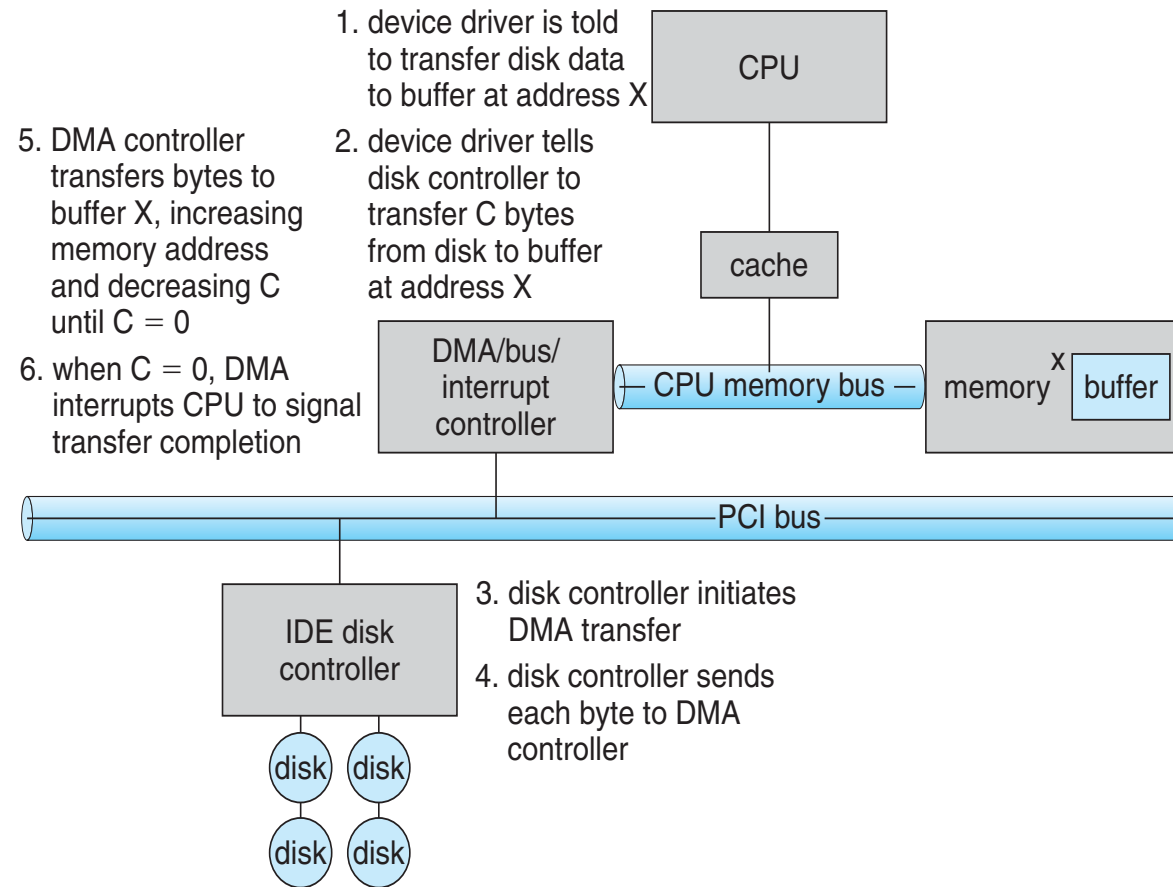
Programmed I/O vs. Memory-Mapped I/O

- Memory-mapped I/O is now dominant for performance.
Example: PCIe BARs (Base Address Registers) expose device memory to CPU address space."
- **Security note:** MMIO can be a vector for attacks (e.g., PCIe DMA attacks). Modern systems use **IOMMU** to restrict device memory access.

More Efficient Data Movement: DMA

- DMA is used to avoid **programmed I/O** for large data movement
 - Programmed I/O (PIO): when CPU is involved in data movement
 - PIO consumes CPU time
 - bypasses CPU to transfer data directly between I/O device and memory
- OS writes DMA command block into memory
 - Source and destination addresses
 - Read or write mode
 - Count of bytes
 - Writes location to DMA controller
 - Bus mastering of DMA controller – grabs bus from CPU
 - **Cycle stealing** from CPU but still much more efficient
 - When done, interrupts to signal completion

More Efficient Data Movement: DMA

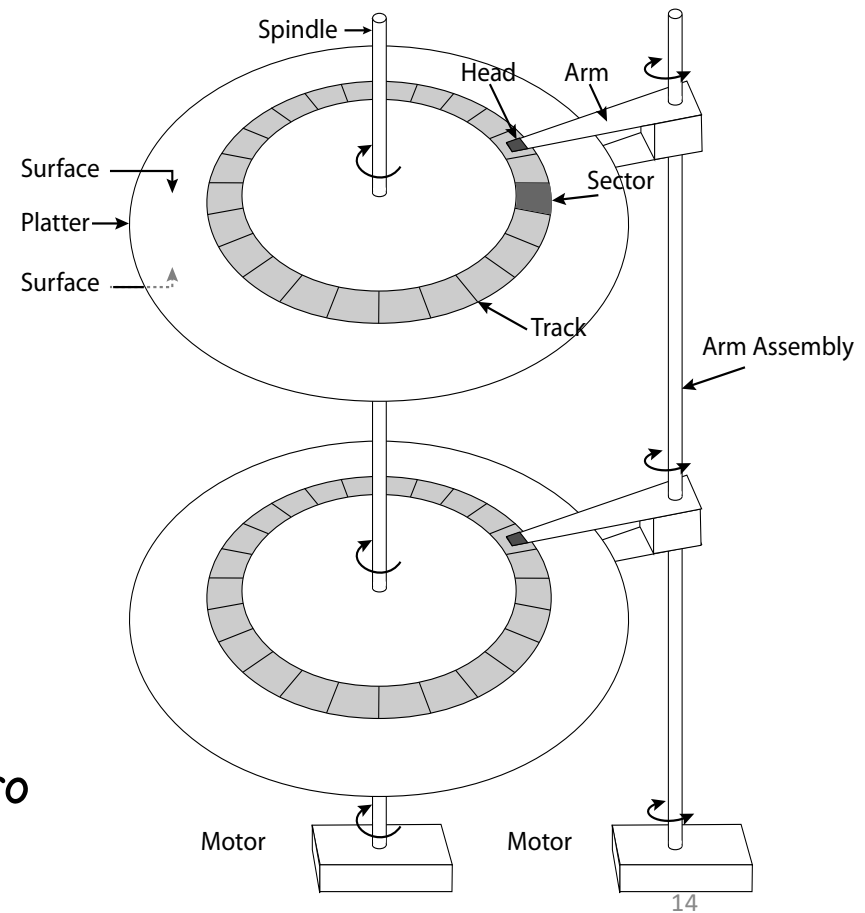


Storage Devices

- Magnetic disks
 - Storage that rarely becomes corrupted
 - Large capacity at low cost
 - Block level random access
 - Poor performance for random access
 - Better performance for sequential access
- Flash memory
 - Storage that rarely becomes corrupted
 - Capacity at intermediate cost (5–20x disk)
 - Block level random access
 - Good performance for reads; worse for random writes

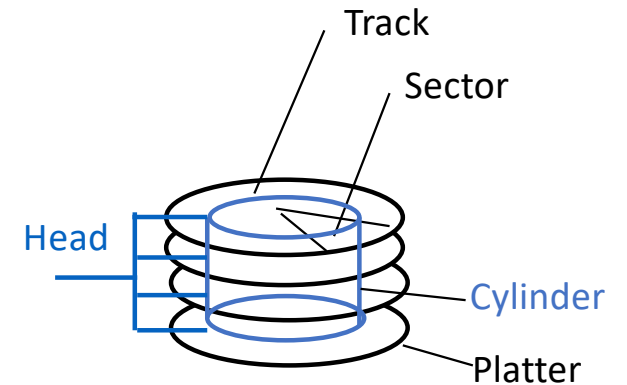
The Amazing Magnetic Disk

- Unit of Transfer: Sector
 - Ring of sectors form a track
 - Stack of tracks form a cylinder
 - Heads position on cylinders
- Disk Tracks $\sim 1\mu\text{m}$ (micron) wide
 - Wavelength of light is $\sim 0.5\mu\text{m}$
 - Resolution of human eye: $50\mu\text{m}$
 - 100K tracks on a typical 2.5" disk
- Separated by unused guard regions
 - Reduces likelihood neighboring tracks are corrupted during writes (still a small non-zero chance)

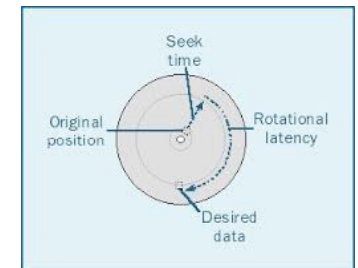


Magnetic Disks

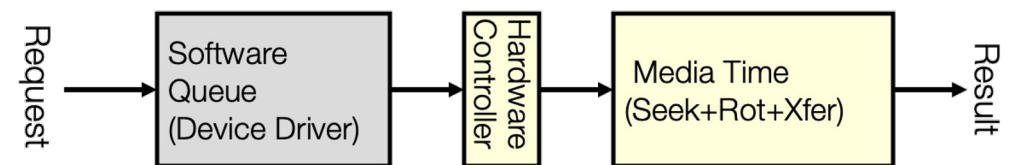
- Hard Disk Drive (HDD)
- Cylinders: all the tracks under the head at a given point on all surface
- Read/write data is a three-stage process:
 - Seek time: position the head/arm over the proper track
 - Rotational latency: wait for desired sector to rotate under r/w head
 - Transfer time: transfer a block of bits (sector) under r/w head



Seek time = 4-8ms
One rotation = 1-2ms
(3600-7200 RPM)



**Disk Latency = Queuing Time + Controller time +
Seek Time + Rotation Time + Transfer Time**



Disk Performance Example

- Assumptions:
 - Ignoring queuing and controller times for now
 - Avg seek time of 5ms,
 - 7200RPM $\Rightarrow$ Time for rotation: $60000 \text{ (ms/minute)} / 7200 \text{ (rev/min)} \cong 8\text{ms}$
 - Transfer rate 4MByte/s, sector size 1 Kbyte $\Rightarrow 1024 \text{ bytes} / 4 \times 10^6 \text{ (bytes/s)} = 256 \times 10^{-6} \text{ sec} \cong .26 \text{ ms}$
- Read sector from random place on disk:
 - Seek (5ms) + Rot. Delay (4ms) + Transfer (0.26ms)
 - *Approx* 10ms to fetch/put data: 100 KByte/sec
- Read sector from random place in the same cylinder:
 - Rot. Delay (4ms) + Transfer (0.26ms)
 - *Approx* 5ms to fetch/put data: 200 KByte/sec
- Read next sector on same track:
 - Transfer (0.26ms): 4 MByte/sec

Key to using disk effectively (especially for file systems) is to *minimize seek and rotational delays*

Typical Numbers for Magnetic Disk

Parameter	Info / Range
Space/Density	Space: 8TB (Seagate), 10TB (Hitachi) in 3½ inch form factor! Areal Density: ≥ 1 Terabit/square inch! (SMR, Helium, ...)
Average seek time	Typically 5-10 milliseconds. Depending on reference locality, actual cost may be 25-33% of this number.
Average rotational latency	Most laptop/desktop disks rotate at 3600-7200 RPM (16-8 ms/rotation). Server disks up to 15,000 RPM. Average latency is halfway around disk so 8-4 milliseconds
Controller time	Depends on controller hardware
Transfer rate	Typically 50 to 100 MB/s.
Cost	Used to drop by a factor of two every 1.5 years (or even faster); now slowing down

Solid State Disks (SSDs)

Component	Description
NAND Flash	Non-volatile memory storing data as trapped charge (electrons)
Cell Types	SLC (1 bit) → MLC (2) → TLC (3) → QLC (4) – cheaper but slower & less durable
3D NAND	Vertical stacking of cells (64–200+ layers); increases density without shrinking cell size
Block/Page Architecture	Read/write in pages (4KB–16KB); erase in blocks (4–64 pages) → <i>write amplification</i>



Key consequence: overwriting requires read-erase-modify cycle

Key Challenges

- **Write endurance** – limited program/erase cycles
 - TLC consumer: ~600 TBW (terabytes written)
 - Enterprise TLC: 1–3 DWPD (drive writes per day)
- **Garbage collection & wear leveling** – firmware must evenly distribute writes
- **Performance consistency** – slows down as drive fills or after sustained writes

HDD vs SSD Comparison

Feature	HDD	SATA SSD	NVMe SSD
Access time	3–10 ms	0.1–0.2 ms	0.02–0.08 ms
Sequential read	150–250 MB/s	500–550 MB/s	3,500–14,000 MB/s
IOPS (rand read)	100–200	10k–100k	200k–1.5M
Price per GB (2025)	~\$0.02	~\$0.05	~\$0.07
Write endurance (TBW)	unlimited	150–600 TBW (consumer)	300–1200 TBW
Power (active/idle)	6W / 3W	3W / 0.5W	5W / 0.8W

Reading Materials

- Three easy pieces, CH36, I/O Devices
 - <https://pages.cs.wisc.edu/~remzi/OSTEP/file-raid.pdf>
- Three easy pieces, CH37, Hard Disk Drives
 - <https://pages.cs.wisc.edu/~remzi/OSTEP/file-disks.pdf>

Thank you!

